

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: Ch.Gouri Sai Sree

Roll No.: A24126552076

Date: 25-08-2026

1. TITLE

Develop a Dynamic To-Do List Application Using JavaScript, DOM Manipulation and Event Listeners

2. AIM

To develop a dynamic To-Do List application using **JavaScript, DOM manipulation, and event listeners** that allows users to add, complete, and delete tasks interactively.

3. OBJECTIVE

The objectives of this experiment are:

- To understand DOM element selection using JavaScript.
 - To dynamically create HTML elements using JavaScript.
 - To modify the DOM dynamically based on user actions.
 - To implement event listeners for handling button clicks.
 - To add and remove task elements from the webpage.
 - To visually indicate completed tasks.
 - To display an appropriate message when the task list is empty.
 - To develop an interactive webpage using JavaScript.
-

4. THEORY

JavaScript is a programming language used to create dynamic and interactive web pages.

DOM (Document Object Model) represents an HTML document as a tree of elements and objects. JavaScript can access and modify these elements dynamically.

Event Listeners allow JavaScript to respond to user actions such as button clicks and checkbox changes.

In this application:

- `document.getElementById()` is used to select HTML elements.
- `document.createElement()` is used to dynamically create task elements.
- `addEventListener()` is used to handle checkbox change events.
- `textContent` is used to display the task text.
- `appendChild()` is used to add dynamically created elements to the task list.
- `style.textDecoration` is used to display a completed task with a strikethrough.
- `style.color` is used to change the appearance of completed tasks.
- `innerHTML` is used to clear all tasks from the list.
- `tasks` array is used to store the entered task values.

The basic flow of the application is:

User Enters Task → JavaScript Processes Input → DOM Creates Task → User Completes/Clears Task → Webpage Updates

5. ALGORITHM / PROCEDURE

1. Create an HTML5 document with a text box for entering tasks.
 2. Add **ADD** and **CLEAR** buttons to the webpage.
 3. Create an unordered list to display the tasks.
 4. Create a JavaScript array to store the entered tasks.
 5. Create the `addTask()` function to handle adding a new task.
 6. Retrieve the task entered by the user using `document.getElementById()`.
 7. Remove unnecessary spaces using `trim()` and prevent empty tasks from being added.
 8. Store the valid task in the `tasks` array.
 9. Dynamically create a `<li>`, checkbox, and `<span>` using `document.createElement()`.
 10. Add an event listener to the checkbox to mark the task as completed.
 11. Use `appendChild()` to add the task elements to the task list.
 12. Clear the input field after adding the task.
 13. Create the `cleartasks()` function to remove all tasks from the list.
 14. Test the application by adding, completing, and clearing different tasks.
 15. Verify that the actual output matches the expected functionality.
-

6. PROGRAM

Todo.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>

  <link rel="stylesheet" href="style.css">
  <script src="javatodo.js" defer></script>
</head>

<body>

  <div class="container">

    <h1>My To-Do List</h1>

    <input type="text" id="taskInput" placeholder="Enter a task...">
    <button id="addTask">Add Task</button>

    <p id="emptyMessage">No tasks available.</p>

    <ul id="taskList"></ul>

  </div>

</body>
</html>
```

Style.css

```
body {
```

```
font-family: Arial, sans-serif;
background-color: #f2f2f2;
text-align: center;
padding: 40px;
}
```

```
.container {
width: 500px;
margin: auto;
background: white;
padding: 25px;
border-radius: 10px;
}
```

```
h1 {
color: #333;
}
```

```
input {
width: 60%;
padding: 10px;
font-size: 16px;
}
```

```
#addTask {
padding: 10px 15px;
background-color: green;
color: white;
border: none;
cursor: pointer;
}
```

```
ul {
padding: 0;
list-style: none;
}
```

```
li {
display: flex;
align-items: center;
justify-content: space-between;
padding: 10px;
margin: 10px 0;
background-color: #eee;
}
```

```

    border-radius: 5px;
}

.task-text {
    flex: 1;
    text-align: left;
}

.completed {
    text-decoration: line-through;
    color: gray;
}

.complete-btn,
.delete-btn {
    margin-left: 5px;
    padding: 6px 10px;
    border: none;
    cursor: pointer;
}

.complete-btn {
    background-color: #2196f3;
    color: white;
}

.delete-btn {
    background-color: red;
    color: white;
}

#emptyMessage {
    color: gray;
}

```

javatodo.js

```

const taskInput = document.getElementById("taskInput");
const addTaskButton = document.getElementById("addTask");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

function updateEmptyMessage() {

```

```
if (taskList.children.length === 0) {  
    emptyMessage.style.display = "block";  
} else {  
    emptyMessage.style.display = "none";  
}  
}
```

```
addTaskButton.addEventListener("click", function () {
```

```
    const taskText = taskInput.value.trim();
```

```
    if (taskText === "") {  
        alert("Please enter a task!");  
        return;  
    }
```

```
    const li = document.createElement("li");
```

```
    const span = document.createElement("span");  
    span.textContent = taskText;  
    span.classList.add("task-text");
```

```
    const completeButton = document.createElement("button");  
    completeButton.textContent = "Complete";  
    completeButton.classList.add("complete-btn");
```

```
    const deleteButton = document.createElement("button");  
    deleteButton.textContent = "Delete";  
    deleteButton.classList.add("delete-btn");
```

```
    completeButton.addEventListener("click", function () {  
        span.classList.toggle("completed");
```

```
        if (span.classList.contains("completed")) {  
            completeButton.textContent = "Completed";  
        } else {  
            completeButton.textContent = "Complete";  
        }  
    });
```

```
    deleteButton.addEventListener("click", function () {  
        li.remove();  
        updateEmptyMessage();
```

```
});  
  
li.appendChild(span);  
li.appendChild(completeButton);  
li.appendChild(deleteButton);  
  
taskList.appendChild(li);  
  
taskInput.value = "";  
  
updateEmptyMessage();  
});  
  
updateEmptyMessage();
```

7. CODE EXPLANATION

Code	Explanation
<code>let tasks = [];</code>	Creates an array to store the entered tasks.
<code>function addTask()</code>	Defines the function used to add a new task.
<code>document.getElementById("todo")</code>	Selects the task input field from the DOM.
<code>.value</code>	Retrieves the task entered by the user.
<code>.trim()</code>	Removes unnecessary spaces from the beginning and end of the task.

<code>if (!taskText) return;</code>	Prevents an empty task from being added.
<code>tasks.push(taskText)</code>	Adds the task to the <code>tasks</code> array.
<code>document.createElement("li")</code>	Dynamically creates a list item for the task.
<code>document.createElement("input")</code>	Dynamically creates an input element for the checkbox.
<code>checkbox.type = "checkbox"</code>	Converts the input element into a checkbox.
<code>document.createElement("span")</code>	Dynamically creates a span element to display the task text.
<code>span.textContent = taskText</code>	Sets the task text inside the span.
<code>checkbox.addEventListener("change", ...)</code>	Detects when the task checkbox is selected or deselected.
<code>span.style.textDecoration</code>	Applies or removes a strikethrough to indicate task completion.
<code>span.style.color</code>	Changes the text color when a task is completed.
<code>li.appendChild(checkbox)</code>	Adds the checkbox to the task item.
<code>li.appendChild(span)</code>	Adds the task text to the task item.
<code>taskList.appendChild(li)</code>	Adds the completed task item to the webpage.
<code>taskInput.value = ""</code>	Clears the input field after adding a task.
<code>taskInput.focus()</code>	Places the cursor back in the task input field.
<code>function cleartasks()</code>	Defines the function used to clear all tasks.
<code>tasks = []</code>	Resets the task array and removes all stored tasks.
<code>taskList.innerHTML = ""</code>	Removes all displayed task elements from the webpage.
<code>onclick="addTask() "</code>	Calls the <code>addTask()</code> function when the ADD button is clicked.
<code>onclick="cleartasks() "</code>	Calls the <code>cleartasks()</code> function when the CLEAR button is clicked.

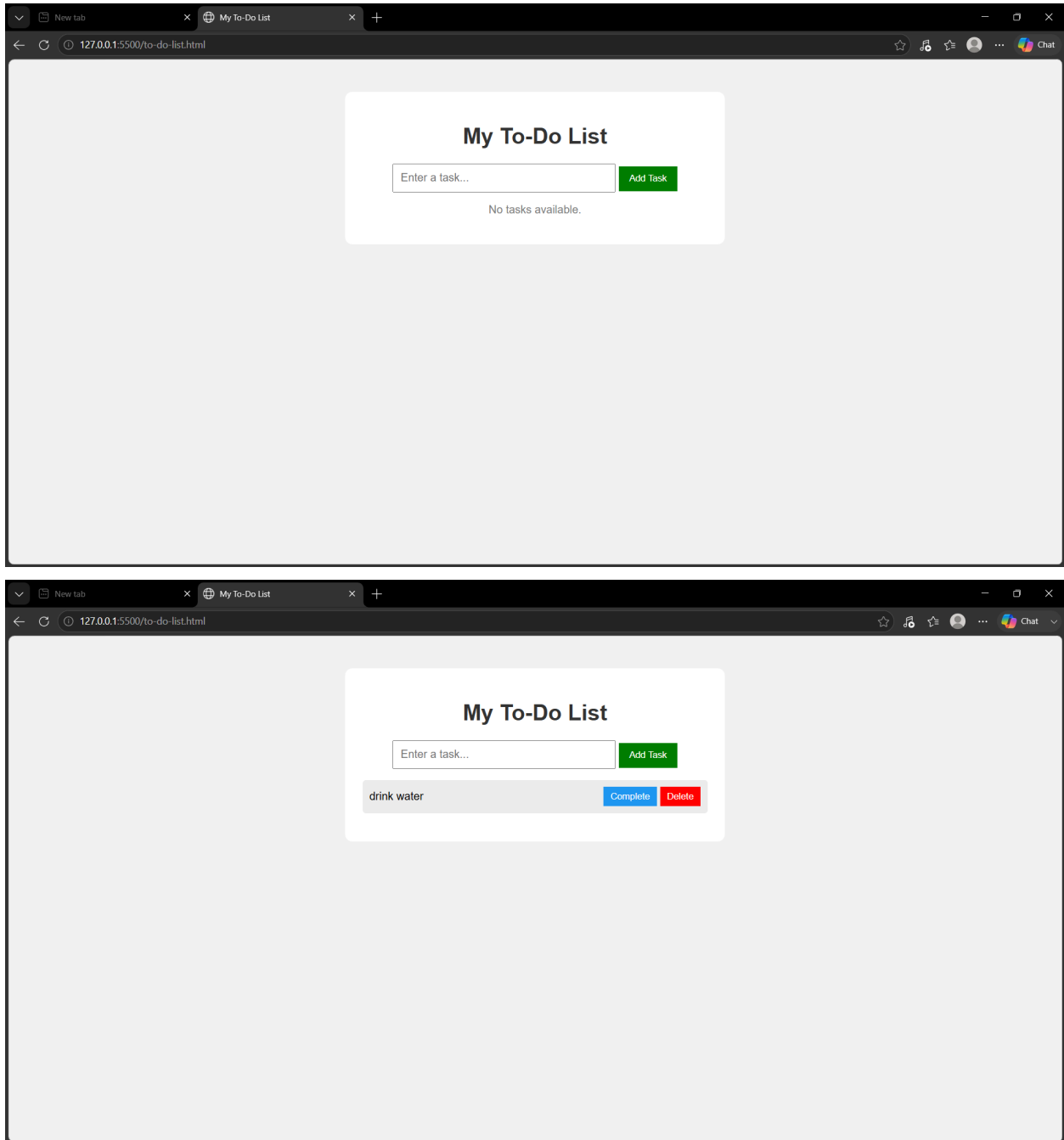
8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

Test Case	Input / Action	Expected Result	Actual Result	Status	Test Case	Input / Action
TC-01	Enter drink water and click ADD	Task should be added to the list	Task added successfully	Pass	TC-01	Enter drink water and click ADD
TC-02	Enter Prepare for Quiz and click ADD	New task should appear below the existing task	Task added successfully	Pass	TC-02	Enter Prepare for Quiz and click ADD
TC-03	Add Practice JavaScript and select its checkbox	Task should be displayed with a strikethrough and changed color	Task marked as completed successfully	Pass	TC-03	Add Practice JavaScript and select its checkbox
TC-04	Click CLEAR after adding multiple tasks	All tasks should be removed from the list	All tasks cleared successfully	Pass	TC-04	Click CLEAR after adding multiple tasks

9. OUTPUT

Output 1: Server Execution



10. RESULT

Thus, a **Dynamic To-Do List application** was successfully developed using **JavaScript, DOM manipulation, and event listeners**. Tasks were dynamically added to the webpage, completed tasks were visually indicated using checkboxes, and all tasks could be cleared from the list. The application was successfully verified using multiple test cases.

